

19. 一个完整算例：从原始回波到热点告警

前面讲了很多概念和算法，现在用一组简化的假数据，把整条链路真正走一遍。你可以把这一节当成“把前面所有内容串起来的练习题”。

这组数据不是某台真实设备的标定结果，所以它不用于科研发表，但非常适合建立工程直觉。

19.1 先设定一个具体场景

假设我们有一台固定式颗粒物 LiDAR，装在工地高点平台上：

1. 设备安装高度为 18 m。
2. 当前在看一个固定方位附近的低仰角扫描。
3. 距离分辨率为 30 m。
4. 我们先只看一条射线上的 6 个 range bin。
5. 然后再把这条射线扩展成一个小扫描扇面，最后提取热点。

为了便于说明，先定义 6 个距离 bin：

$$R = [30, 60, 90, 120, 150, 180] \text{ m}$$

原始 photon counts 假设如下：

$$P_{\text{raw}} = [34, 38, 55, 110, 82, 45]$$

同时假设：

1. 远端背景均值 $B = 20 \text{ counts}$ 。
2. 这一帧的激光单脉冲能量 $E = 2.0 \text{ mJ}$ 。
3. overlap 曲线在近距离不完整：

$$O(R) = [0.40, 0.70, 0.90, 1.00, 1.00, 1.00]$$

19.2 第一步：背景扣除

背景扣除公式是：

$$P_1(R) = P_{\text{raw}}(R) - B$$

带入这组数据后：

$$P_1 = [14, 18, 35, 90, 62, 25]$$

这里最值得注意的是：

1. 原始 34 counts 并不表示有 34 counts 都来自目标。
2. 其中有 20 counts 只是背景底噪。
3. 真正属于目标回波的只有扣背景之后的部分。

你可以把这一步理解成：

先把房间里的环境光关掉，再看手电筒真正照到了什么。

19.3 第二步：发射能量归一化

能量归一化公式是：

$$P_2(R) = \frac{P_1(R)}{E}$$

这里的 E 就是这个 profile 对应的发射激光能量。更准确地说：

E 是这一枪，或者这一帧平均意义下，实际发出去的激光脉冲能量。

它不是接收端收到的光能量，而是发射端发出去的能量。因为回波信号 $P_1(R)$ 会随着发射能量一起变大或变小，所以要先除以 E ，把它变成“单位发射能量下的回波强度”。

带入 $E = 2.0 \text{ mJ}$ 后：

$$P_2 = [7.00, 9.00, 17.50, 45.00, 31.00, 12.50]$$

单位你可以先理解成“每毫焦对应的计数”。

再用一个单点例子看得更清楚：

1. 同一片空气，第一帧发射能量是 2.0 mJ，回波是 100 counts。
2. 第二帧发射能量是 1.8 mJ，回波是 90 counts。
3. 直接看 100 和 90，好像空气变弱了。
4. 但做能量归一化后，二者都是 50 counts/mJ。

这说明变化来自激光器发射能量，而不是来自空气中的颗粒物变化。

为什么这一步不能省：

1. 因为不同脉冲的发射能量会抖动。
2. 不归一化的话，两帧之间的差别不一定来自空气变化，也可能只是激光器输出变强或变弱。
3. 归一化之后，后面的 RCS、overlap 校正和反演才是在比较空气本身，而不是比较激光器哪一枪更有劲。

19.4 第三步：RCS 距离平方校正

距离平方校正公式：

$$RCS(R) = P_2(R)R^2$$

计算后得到：

$$RCS = [6300, 32400, 141750, 648000, 697500, 405000]$$

这一步先把“远处天然更暗”的几何趋势补回来。注意，它还没有处理近距离 overlap 不完整的问题，所以近端数值仍然可能偏低。

19.5 第四步：overlap 校正

现在把近距离重叠不完整的问题补回来：

$$P_{corr}(R) = \frac{RCS(R)}{O(R)}$$

把每个距离 bin 算出来：

距离 R (m)	原始计数 Praw	扣背景后 P1	能量归一化 P2	RCS	overlap 校正后 Pcorr
30	34	14	7.00	6300	15750
60	38	18	9.00	32400	46286
90	55	35	17.50	141750	157500
120	110	90	45.00	648000	648000
150	82	62	31.00	697500	697500
180	45	25	12.50	405000	405000

这一步有一个很重要的现象：

- 1. 30 m 处的 RCS 本来只有 6300。
- 2. 但因为 overlap 只有 0.40，校正后变成了 15750。

这说明近距离如果不做 overlap 校正，近端粉尘会被严重低估。

说明：RCS 和 overlap 校正都是逐距离点的乘除操作，数学上先后可交换。本文为了和第 11 章预处理流程一致，统一写成“先 RCS，再除以 $O(R)$ ”。

从这个表里你可以立刻看到两件事：

- 1. 120 m 到 150 m 之间是这条射线里最显著的高回波区。
- 2. 150 m 的 RCS 比 120 m 还高，说明那里可能仍然存在明显颗粒物结构。

但注意，这里还不能直接说 “150 m 的 PM 一定最高” ，因为传播衰减还没有真正被反演掉。

19.6 第五步：给 Klett/Fernald 反演准备边界条件

现在开始做反演。我们先给出一个工程上常见的简化假设：

- 1. 取 lidar ratio $S = 45$ sr。
- 2. 取最远端 180 m 作为参考 bin。
- 3. 假设参考端消光系数：

$$\alpha_{\text{ref}} = 0.18 \text{ km}^{-1}$$

这相当于告诉算法：

最远端那一格我先给你一个可接受的起点，你从这里往回推。

19.7 第六步：做一个简化版 Klett 反演

为了便于教学，我们直接给出这组假数据经过简化递推后的结果。你可以把它理解成 “程序已经根据 RCS、边界条件和 lidar ratio 算好的输出” 。

距离 R (m)	backscatter β ($\text{km}^{-1} \text{ sr}^{-1}$)	extinction $\alpha = S\beta$ (km^{-1})
30	0.0019	0.086
60	0.0022	0.099
90	0.0030	0.135
120	0.0063	0.284
150	0.0051	0.230
180	0.0040	0.180

这里的结果比单看 RCS 更有物理意义，因为它试图把双程传播衰减的影响部分剥离掉。

从结果上看：

- 1. 120 m 的消光系数最高。
- 2. 150 m 也仍然偏高。

3. 说明这条射线上真正的主污染区大致位于 120 m 到 150 m。

这就是为什么：

你不能只看 RCS 或颜色深浅，而必须通过反演去判断真正的光学浓度结构。

19.8 第七步：加入湿度修正

现在假设这条路径上的相对湿度随距离略有变化：

$$RH = [72, 74, 76, 78, 80, 82] \%$$

并且取吸湿增长经验参数：

$$\gamma = 0.45$$

湿度修正公式：

$$\alpha_{\text{dry}}(R) = \alpha(R) \cdot \left[1 - \frac{RH(R)}{100}\right]^\gamma$$

计算后得到：

距离 R (m)	原始消光 α	RH (%)	干态消光 α_{dry}
30	0.086	72	0.048
60	0.099	74	0.054
90	0.135	76	0.071
120	0.284	78	0.144
150	0.230	80	0.112
180	0.180	82	0.083

这个表最值得你注意的点是：

- 1. 所有点的消光都下降了。
- 2. 120 m 仍然是最高值。
- 3. 说明它的高值不只是“湿度太大造成的虚高”，而是真有明显颗粒物结构。

19.9 第八步：把干态消光转成 PM2.5 / PM10

为了教学简单，我们先用一个非常朴素的线性模型：

$$PM_{2.5} = 500 \cdot \alpha_{dry} + 10$$

再假设：

$$PM_{10} = 1.6 \cdot PM_{2.5}$$

注意：这只是教学用经验关系，真实项目里必须用本地标定模型替代。

带入后得到：

距离 R (m)	干态消光 α_{dry}	PM2.5 ($\mu g/m^3$)	PM10 ($\mu g/m^3$)
30	0.048	34.0	54.4
60	0.054	37.0	59.2
90	0.071	45.5	72.8
120	0.144	82.0	131.2
150	0.112	66.0	105.6
180	0.083	51.5	82.4

如果我们把热点阈值设成：

$$PM_{2.5} > 60 \mu g/m^3$$

那么这条射线上会有两个连续高值 bin：

1. 120 m。
2. 150 m。

到这里为止，你已经从一条原始回波数组，走到了“这条射线上的高污染区在哪里”。

19.10 第九步：把单条射线扩展成一个小扫描扇面

现在我们不只看一条射线，而是假设相邻两个方位角也扫到了类似高值。为了简化说明，只取 4 个热点 点元：

点编号	方位角 (deg)	仰角 (deg)	距离 R (m)	PM2.5
A	40	10	120	82
B	40	10	150	66
C	42	10	120	79
D	42	10	150	63

这些点说明：

- 1. 高值区不只出现在一条射线上。
- 2. 它已经在相邻扫描角里形成了一个连续的二维小斑块。
- 3. 这就具备了“提取热点区域”的基础。

19.11 第十步：把热点元转成 ENU 坐标

仍然使用固定式系统的 ENU 公式：

$$\begin{aligned}x &= R \cos(\text{elevation}) \sin(\text{azimuth}) \\y &= R \cos(\text{elevation}) \cos(\text{azimuth}) \\z &= R \sin(\text{elevation})\end{aligned}$$

算出来的近似结果如下：

点编号	ENU x (m)	ENU y (m)	ENU z (m)	权重 PM2.5
A	75.9	90.5	20.8	82
B	94.9	113.2	26.0	66
C	79.0	87.8	20.8	79
D	98.8	109.7	26.0	63

此时这些点已经不是抽象的“第几个 bin”，而是真正的空间点。

19.12 第十一步：计算热点质心

如果用 PM2.5 做权重，那么热点加权质心公式是：

$$x_c = \frac{\sum_i w_i x_i}{\sum_i w_i}, \quad y_c = \frac{\sum_i w_i y_i}{\sum_i w_i}, \quad z_c = \frac{\sum_i w_i z_i}{\sum_i w_i}$$

带入 A、B、C、D 四个点之后，可以得到近似质心：

$$(x_c, y_c, z_c) \approx (86.1, 99.1, 23.2) \text{ m}$$

这意味着什么？

意思是如果你要驱动一个喷雾炮去瞄准这个热点，那么它最值得优先瞄准的位置，不是 A、B、C、D 某一个点，而是这个加权质心附近。

19.13 第十二步：把质心转成控制目标

对于固定式喷雾炮，平台通常还要继续算两个量：

1. 水平距离。
2. 控制方位角和俯仰角。

水平距离：

$$R_{\text{horizontal}} = \sqrt{x_c^2 + y_c^2} \approx 131.2 \text{ m}$$

目标方位角近似为：

$$\text{heta} = \arctan\left(\frac{x_c}{y_c}\right) \approx 41^\circ$$

目标仰角近似为：

$$\phi = \arctan\left(\frac{z_c}{R_{\text{horizontal}}}\right) \approx 10^\circ$$

这个结果很合理，因为它正好落在前面 40 到 42 度、10 度仰角那一小片高值区中间。

19.14 第十三步：形成热点事件记录

到平台层，通常会把它整理成一条可消费事件：

```
{
  "event_id": "dust_demo_001",
  "timestamp": "2026-05-27T10:23:15Z",
  "type": "dust_hotspot",
  "center_enu_m": [86.1, 99.1, 23.2],
  "target_azimuth_deg": 41.0,
  "target_elevation_deg": 10.0,
  "peak_pm25_ugm3": 82.0,
  "mean_pm25_ugm3": 72.5,
  "recommended_action": "spray"
}
```

此时这条数据已经非常适合：

1. 给前端画告警卡片。
2. 给地图画一个热点标记。
3. 给喷雾联动模块发控制指令。
4. 给报表系统留痕。

19.15 用一段代码把这个算例串起来

下面是一段非常简化但逻辑完整的示意代码，用来把这一节的算例串起来：

```
import numpy as np

ranges = np.array([30, 60, 90, 120, 150, 180], dtype=float)
raw_counts = np.array([34, 38, 55, 110, 82, 45], dtype=float)
background = 20.0
laser_energy = 2.0
overlap = np.array([0.40, 0.70, 0.90, 1.00, 1.00, 1.00], dtype=float)
rh = np.array([72, 74, 76, 78, 80, 82], dtype=float)

signal_bg = raw_counts - background
signal_norm = signal_bg / laser_energy
signal_overlap = signal_norm / overlap
rcs = signal_overlap * ranges**2

beta = np.array([0.0019, 0.0022, 0.0030, 0.0063, 0.0051, 0.0040])
alpha = 45.0 * beta
alpha_dry = alpha * (1.0 - rh / 100.0) ** 0.45

pm25 = 500.0 * alpha_dry + 10.0
pm10 = 1.6 * pm25

hot_mask = pm25 > 60.0
hot_ranges = ranges[hot_mask]
hot_pm25 = pm25[hot_mask]

print("RCS:", np.round(rcs, 1))
print("alpha:", np.round(alpha, 3))
print("PM2.5:", np.round(pm25, 1))
print("热点距离:", hot_ranges)
print("热点PM2.5:", np.round(hot_pm25, 1))
```

你可以看到，这段代码虽然不长，但它已经完整体现了这条思路：

1. 原始回波先做清洗。
2. 再做 RCS。
3. 再做反演输出光学量。
4. 再做湿度修正。
5. 再做 PM 推断。
6. 最后再做热点判别。

19.16 这个完整算例真正想让你学会什么

如果你把这节真正看懂了，其实就已经建立了一个很重要的工程直觉：

1. 原始回波不是业务结果，它只是起点。
2. 预处理不是锦上添花，而是决定后面会不会全盘跑偏。
3. 反演解决的是“把回波变成光学量”的问题。
4. 湿度修正和地面标定解决的是“把光学量变成 PM”的问题。
5. ENU 映射和热点提取解决的是“把 PM 场变成可行动坐标”的问题。
6. 最终平台真正消费的，不是一条波形，而是一条结构化事件。

如果你愿意，下一步完全可以把这一节再继续升级成：

1. 带真实 Python 可运行脚本的版本。
2. 带图表输出的 notebook 版本。
3. 带 RHI 或 PPI 小矩阵示例的二维版本。

19.17 再进一步：二维 PPI 扫描面小矩阵算例

上面的例子本质上还是“少量热点单元”。现在我们把它再往前推进一步，真正做成一个二维扫描面。

这里选择 PPI 作为例子，因为它更贴近工地和道路场景里最常见的“水平扩散监控”。

PPI 的意思是：

1. 仰角固定。
2. 方位角不断变化。
3. 每个方位角上都有一串距离 bin。

所以 PPI 的数据天然就是一个二维矩阵：

$$PM[\text{azimuth}, R]$$

你可以把它理解成：

每一行是一条射线，每一列是这条射线上的一个距离格子，整张表拼起来就是一个扫描扇面。

19.18 先定义一个最小 PPI 小矩阵

假设：

1. 固定仰角为 10° 。
2. 方位角一共扫 3 条： 38° 、 40° 、 42° 。
3. 距离 bin 取 4 个：60 m、90 m、120 m、150 m。
4. 每个格子的 PM_{2.5} 都已经经过前面的整条算法链，也就是已经完成了预处理、反演和湿度修正后的标定输出。

得到的二维 PM2.5 小矩阵如下：

方位角 距离	60 m	90 m	120 m	150 m
38 deg	34	46	78	58
40 deg	37	51	82	66
42 deg	35	49	79	63

读这个表时，你要这样看：

- 1. 同一行表示同一条扫描射线。
- 2. 同一列表示相近的距离圈层。
- 3. 右上区域数值更高，说明热点不是单点，而是一个小片区。

19.19 第一步：做阈值分割

仍然使用前面的业务阈值：

$$PM_{2.5} > 60 \mu g/m^3$$

把矩阵转成二值 mask 后：

方位角 距离	60 m	90 m	120 m	150 m
38 deg	0	0	1	0
40 deg	0	0	1	1
42 deg	0	0	1	1

这里的 1 表示热点候选单元，0 表示非热点单元。

从这个二值表里，你已经能直观看见一件事：

- 1. 高值区主要集中在 120 m，部分延伸到 150 m。
- 2. 并且跨越了多个相邻方位角。
- 3. 这说明它不是随机噪点，而是一个连续粉尘斑块。

19.20 第二步：连通域判断这个热点是不是一个整体

对二维扫描面来说，阈值分割之后通常还要做连通域分析。

在这个小例子里，值为 1 的格子共有 5 个：

- 1. 38 deg, 120 m。

- 2. 40 deg, 120 m。
- 3. 40 deg, 150 m。
- 4. 42 deg, 120 m。
- 5. 42 deg, 150 m。

如果采用 8 邻域连通规则，这 5 个格子会被识别为同一个热点区域，而不是 5 个彼此无关的小点。
这一步非常关键，因为平台真正关心的是：

这里是不是存在一个连续污染团，而不是某一个像素偶然超阈值。

19.21 第三步：把二维热点格子转成空间坐标

现在把这 5 个热点格子真正转换到 ENU 坐标。

固定仰角仍然是 10°，转换公式与前面一致：

$$\begin{aligned}x &= R \cos(10^\circ) \sin(\text{azimuth}) \\y &= R \cos(10^\circ) \cos(\text{azimuth}) \\z &= R \sin(10^\circ)\end{aligned}$$

算出来的热点格子中心坐标近似如下：

热点格子	PM2.5	x (m)	y (m)	z (m)
38 deg, 120 m	78	72.8	93.1	20.8
40 deg, 120 m	82	76.0	90.5	20.8
42 deg, 120 m	79	79.0	87.8	20.8
40 deg, 150 m	66	94.9	113.2	26.0
42 deg, 150 m	63	98.8	109.7	26.0

到了这里，二维扫描面上的 “5 个高值格子” 就已经被转换成了 “5 个真实空间点” 。

19.22 第四步：估算二维热点的加权质心

现在用 PM2.5 作为权重，计算这 5 个热点单元的加权质心。

公式和前面一样：

$$x_c = \frac{\sum_i w_i x_i}{\sum_i w_i}, \quad y_c = \frac{\sum_i w_i y_i}{\sum_i w_i}, \quad z_c = \frac{\sum_i w_i z_i}{\sum_i w_i}$$

把这 5 个单元代入之后，可以得到近似结果：

$$(x_c, y_c, z_c) \approx (83.3, 97.9, 22.7) \text{ m}$$

这个结果怎么理解：

1. 这条剖面里的主要污染团，平均位于设备前方约 130 m。
2. 它的主质量中心大约在 23 m 高度。
3. 如果你要做平面热点研判，这个质心比盯着矩阵本身更有用。

19.23 第五步：把二维热点质心转成喷雾目标角

有了质心坐标之后，控制系统常要继续算：

1. 水平距离。
2. 目标方位角。
3. 目标仰角。

水平距离：

$$R_{\text{horizontal}} = \sqrt{x_c^2 + y_c^2} \approx 128.5 \text{ m}$$

目标方位角近似为：

$$\text{az}_{\text{target}} = \arctan\left(\frac{x_c}{y_c}\right) \approx 40.4^\circ$$

目标仰角近似为：

$$\phi_{\text{target}} = \arctan\left(\frac{z_c}{R_{\text{horizontal}}}\right) \approx 10.0^\circ$$

从这个结果你可以看到，二维扫描面算出来的热点目标角，仍然稳定落在 40 度左右、10 度左右，这和我们一开始的人眼直观判断是一致的。

19.24 第六步：如果把这个二维结果画到 UI 上，会长什么样

在 PPI 页面上，它不会表现成 5 个孤立数字，而会更像一个扇形网格中的连续高值区。

你可以把它抽象成下面这样：

PPI fixed elevation = 10 deg

azimuth \ range 60m 90m 120m 150m

```
38 deg      low  low  high  low
40 deg      low  low  high  high
42 deg      low  low  high  high
```

```
centroid -> (83.3, 97.9, 22.7) m
```

```
target    -> az=40.4 deg, el=10.0 deg
```

真正的前端显示里，通常会同时叠加：

1. 热力图颜色。
2. 阈值轮廓线。
3. 热点质心点。
4. 当前建议喷雾指向。

这时候值班人员看到的就不再是抽象矩阵，而是一张可直接决策的业务图。

19.25 第七步：用代码把这个二维算例串起来

下面这段示意代码把二维 PPI 小矩阵的核心步骤连起来：

```
import numpy as np

azimuths = np.array([38.0, 40.0, 42.0], dtype=float)
ranges = np.array([60.0, 90.0, 120.0, 150.0], dtype=float)
elevation = 10.0

pm25 = np.array([
    [34.0, 46.0, 78.0, 58.0],
    [37.0, 51.0, 82.0, 66.0],
    [35.0, 49.0, 79.0, 63.0],
])

mask = pm25 > 60.0
section_points = []

for iaz, azimuth in enumerate(azimuths):
    az = np.deg2rad(azimuth)
    el = np.deg2rad(elevation)

    for ir, radius in enumerate(ranges):
        if not mask[iaz, ir]:
            continue

        value = pm25[iaz, ir]
        x = radius * np.cos(el) * np.sin(az)
        y = radius * np.cos(el) * np.cos(az)
        z = radius * np.sin(el)
        section_points.append((value, x, y, z))
```

```
weights = np.array([row[0] for row in section_points])
xyz = np.array([[row[1], row[2], row[3]] for row in section_points])
centroid = (weights[:, None] * xyz).sum(axis=0) / weights.sum()

horizontal_range = np.hypot(centroid[0], centroid[1])
target_azimuth = np.rad2deg(np.arctan2(centroid[0], centroid[1]))
target_elevation = np.rad2deg(np.arctan2(centroid[2], horizontal_range))

print("hotspot_cell_count =", len(section_points))
print("centroid_enu_m =", np.round(centroid, 2))
print("target_az_el_deg =", np.round([target_azimuth, target_elevation], 2))
```

这段代码体现的是二维扫描面的标准流程：

1. 二维矩阵先做阈值筛选。
2. 保留热点单元。
3. 把热点单元逐个转成三维坐标。
4. 最后再算质心和控制目标。

19.27 这个二维算例真正想让你明白什么

这一节最重要的，不是记住某个数字，而是建立下面这套直觉：

1. 一维射线算例解决的是“单条光路怎么看”。
2. 二维 PPI 算例解决的是“一个扫描面怎么看”。
3. 真正的业务热点，不是单个 bin，而是二维或三维上的连续区域。
4. 平台做告警和联动时，核心对象不是波形，而是区域、质心、面积和目标角。

19.28 再补一个垂直剖面例子：RHI 到底怎么看

前面的 PPI 解决的是“热点在平面上扩到哪里去了”。

RHI 解决的则是另外一个非常关键的问题：

这个污染羽流抬升到了多高、层底层顶在哪里、是不是正在往上翻卷。

RHI 的含义是：

1. 方位角固定。
2. 仰角不断变化。
3. 在一个固定方向上把空间剖开，得到一张“距离-高度”剖面图。

对工地扬尘、料场扬尘、烟羽抬升判断来说，RHI 特别有用，因为它能直接告诉你：

1. 粉尘是贴地扩散，还是已经被抬到半空。
2. 热点的垂直厚度有多大。
3. 喷雾是该打近地面，还是该打更高的位置。

19.29 先定义一个最小 RHI 小矩阵

假设：

1. 固定方位角为 40 deg。
2. 设备安装高度为 18 m。
3. 仰角扫 4 条：4 deg、8 deg、12 deg、16 deg。
4. 距离 bin 取 4 个：60 m、90 m、120 m、150 m。
5. 每个格子的值依然是假设已经经过完整算法链输出的 PM2.5。

于是得到一个二维 RHI 小矩阵：

仰角 距离	60 m	90 m	120 m	150 m
4 deg	28	45	72	65
8 deg	32	58	95	84
12 deg	30	55	88	76
16 deg	24	41	62	54

你可以先直接目测这张表：

1. 主要高值集中在 120 m 到 150 m。
2. 高值覆盖了多个仰角。
3. 说明这不是地面一条很薄的带，而是一个向上抬起的垂直污染羽流。

19.30 第一步：做阈值分割

还是沿用前面的业务阈值：

$$PM_{2.5} > 60 \mu g/m^3$$

转成二值 mask 之后：

仰角 距离	60 m	90 m	120 m	150 m
4 deg	0	0	1	1

仰角 距离	60 m	90 m	120 m	150 m
8 deg	0	0	1	1
12 deg	0	0	1	1
16 deg	0	0	1	0

这个结果说明：

1. 热点主要存在于 120 m 到 150 m 这一带。
2. 从 4 deg 一直延伸到 16 deg。
3. 所以它不是单点，而是一条有厚度的竖向污染剖面。

19.31 第二步：把 RHI 格子转成地距-高度坐标

对固定方位角的 RHI 来说，最常用的不是先转完整三维，而是先转成剖面坐标：

1. 地面投影距离 s 。
2. 高度 h 。

公式是：

$$s = R \cos(\text{elevation})$$

$$h = h_0 + R \sin(\text{elevation})$$

其中：

1. $h_0 = 18 \text{ m}$ 是设备安装高度。
2. s 表示沿这个固定方位方向，离设备有多远。
3. h 表示这个格子真正处在多高的位置。

把所有热点格子换算后，得到近似结果：

热点格子	PM2.5	地距 s (m)	高度 h (m)
4 deg, 120 m	72	119.7	26.4
4 deg, 150 m	65	149.6	28.5
8 deg, 120 m	95	118.8	34.7
8 deg, 150 m	84	148.5	38.9
12 deg, 120 m	88	117.4	42.9

热点格子	PM2.5	地距 s (m)	高度 h (m)
12 deg, 150 m	76	146.7	49.2
16 deg, 120 m	62	115.4	51.1

这张表特别重要，因为到这一步你终于能回答：

1. 污染并不是只在地面附近。
2. 它已经从大约 26 m 一直抬升到 51 m 左右。
3. 这说明这是一个有明显竖向结构的羽流，而不是单纯贴地积尘。

19.32 第三步：估算层底、层顶和剖面厚度

RHI 最常见的一个业务动作，就是从热点区里估算：

1. 层底高度。
2. 层顶高度。
3. 垂直厚度。

对这组数据：

$$h_{\text{bottom}} \approx 26.4 \text{ m}$$

$$h_{\text{top}} \approx 51.1 \text{ m}$$

所以剖面厚度约为：

$$\text{depth}_{\text{vertical}} = h_{\text{top}} - h_{\text{bottom}} \approx 24.7 \text{ m}$$

这一组量在业务里非常有价值，因为它直接回答了：

1. 羽流最高抬到了多高。
2. 近地喷雾能不能覆盖到它。
3. 这个污染区是“薄薄一层”还是“厚厚一团”。

19.33 第四步：计算 RHI 剖面的加权质心

如果用 PM2.5 作为权重，那么在 RHI 平面里我们先算一个剖面质心：

$$s_c = \frac{\sum_i w_i s_i}{\sum_i w_i}, \quad h_c = \frac{\sum_i w_i h_i}{\sum_i w_i}$$

把上面 7 个热点格子代入之后，可得到近似结果：

$$(s_c, h_c) \approx (130.5, 38.7) \text{ m}$$

这个结果怎么理解：

1. 这条剖面里的主要污染团，平均位于设备前方约 130 m。
2. 它的主质量中心大约在 39 m 高度。
3. 如果你要做垂直剖面的人工研判，这两个数比盯着矩阵本身更有用。

19.34 第五步：如果还要转回三维 ENU 怎么做

RHI 只是固定在一个方位角上的二维切片。如果还要把剖面质心放回三维地图，则需要再把固定方位角 40 deg 代入：

$$\begin{aligned} x_c &= s_c \sin(40^\circ) \\ y_c &= s_c \cos(40^\circ) \\ z_c &= h_c \end{aligned}$$

得到近似三维质心：

$$(x_c, y_c, z_c) \approx (83.9, 100.0, 38.7) \text{ m}$$

和前面的 PPI 热点相比，你会发现：

1. 水平位置仍然接近同一片区域。
2. 但这次额外知道了它在竖直方向上的主中心高度更高。
3. 这就是为什么 PPI 和 RHI 最好配合使用，而不是只用其中一种。

19.35 第六步：如果把这个 RHI 结果画到 UI 上，会长什么样

在 RHI 页面上，它更像下面这种“地距-高度”结构：

```
RHI fixed azimuth = 40 deg

elevation \ range   60m   90m   120m  150m
4 deg          low    low    high   high
8 deg          low    low    high   high
12 deg         low    low    high   high
16 deg         low    low    high   low

section centroid -> ground_range=130.5 m, height=38.7 m
layer bottom     -> 26.4 m
layer top        -> 51.1 m
```

前端真正展示时，通常会把它画成：

1. 距离-高度热力图。
2. 叠加热点轮廓线。
3. 叠加层顶、层底和质心标记。
4. 必要时叠加风向或喷雾可达范围。

19.36 第七步：用代码把这个 RHI 算例串起来

下面是一段示意代码，把二维 RHI 小矩阵从阈值筛选一路串到剖面质心：

```
import numpy as np

elevations = np.array([4.0, 8.0, 12.0, 16.0], dtype=float)
ranges = np.array([60.0, 90.0, 120.0, 150.0], dtype=float)
sensor_height = 18.0

pm25 = np.array([
    [28.0, 45.0, 72.0, 65.0],
    [32.0, 58.0, 95.0, 84.0],
    [30.0, 55.0, 88.0, 76.0],
    [24.0, 41.0, 62.0, 54.0],
])

mask = pm25 > 60.0
section_points = []

for iel, elevation in enumerate(elevations):
    el = np.deg2rad(elevation)

    for ir, radius in enumerate(ranges):
        if not mask[iel, ir]:
            continue

        value = pm25[iel, ir]
        ground_range = radius * np.cos(el)
        height = sensor_height + radius * np.sin(el)
        section_points.append((value, ground_range, height))

weights = np.array([row[0] for row in section_points])
sh = np.array([[row[1], row[2]] for row in section_points])
centroid = (weights[:, None] * sh).sum(axis=0) / weights.sum()

layer_bottom = sh[:, 1].min()
layer_top = sh[:, 1].max()

print("section_point_count =", len(section_points))
print("section_centroid_sh =", np.round(centroid, 2))
print("layer_bottom_top =", np.round([layer_bottom, layer_top], 2))
```

这段代码的逻辑很直接：

1. 先在二维 RHI 矩阵上做阈值筛选。
2. 再把超阈值格子转成剖面坐标。
3. 再从这些剖面点里提取层底、层顶和质心。

19.37 这个 RHI 算例真正想让你学会什么

这一节真正想让你建立的是下面这套判断框架：

1. PPI 更擅长看水平扩散和热点平面位置。
 2. RHI 更擅长看垂直结构、抬升高度和层顶层底。
 3. 如果只看 PPI，你知道热点在哪，但不一定知道它抬多高。
 4. 如果只看 RHI，你知道它抬多高，但不一定知道它横向扩到哪。
 5. 真正成熟的业务系统，往往需要 PPI 和 RHI 配合，才能同时支持告警、研判和联动控制。
-